CAPE PENINSULA UNIVERSITY OF TECHNOLOGY

Marks Administration System

What the application does, how far it is, and the issues we fixed and still have

Group 21 · SDN260S Software Design II · 23 October 2026

This is the console demonstration for the current hand-in, and it is also the honest status of the whole build. The Swing screens are in the source. They are not part of the demonstration. The same console session is in `Group21_CLI_Demo.mp4`.

1. What the application is

The Marks Administration System is a desktop tool for lecturers and a head of department at CPUT Bellville. A lecturer keeps a class list, defines a module and its assessment weights, types marks, and gets a weighted total and a letter grade. The same person can import or export a CSV file and print a student report, a subject report, or a list of missing marks. A head of department can do all of that and can also add, edit and delete user accounts.

There are two ways to drive it. `App` asks for 1 (console) or 2 (Swing). Both talk to the same services and the same SQLite file, so a mark saved in the console is still there when the program starts again. The database file defaults to `data/marks.db`. The demo used `/tmp/demo.db` so the project file was left alone.

Who can do what

Action	Lecturer	Head of department
Log in	Yes. lecturer1 / pass123	Yes. hod1 / admin123
Students, subjects, marks	Add, edit, delete, search	Same
Weighted total, letter grade, GPA	Calculated on save	Same
Reports and missing-mark list	Yes	Yes
CSV template, import, export	Yes	Yes
User accounts	Menu option is absent	Option 6, User Management

What is stored

Six tables: `students`, `subjects`, `subject_assessments` (the weight of each test or exam), `marks` (one row per student, subject and assessment), `users`, and `settings`. Passwords are PBKDF2 hashes. A first run seeds seven CPUT Bellville engineering programmes, 31 modules, 23 students with nine-digit numbers, and example marks. The seed runs once. Deleting marks later does not bring the example data back, because `settings.seeded` is set.

Grades use the thresholds in `Mark.Grade`: A+ from 90, A from 85, B+ from 80, B from 75, C+ from 70, C from 65, D from 60, and F below that. The weighted total uses every assessment on the subject. A blank score is missing: it adds zero and the flag is set, so skipping the exam cannot raise the result. GPA is on a 4.00 scale from those letter grades.

How the code is arranged

The console never contains SQL, CSV parsing or password hashing. `CLIMain` calls services. `MarkService` and `UserService` call `MarkRepository`. `Database` is the SQLite class that implements that interface. `UserService` implements `AuthenticationService`, and the console depends on the interface. Public classes and the business methods have Javadoc. Model getters are left uncommented because the names are enough. Twenty-eight JUnit tests cover grades, hashes, save-and-reload, CSV and reports.

2. How far we are

For this hand-in the brief asks for a console prototype, at least 40 to 50 percent of the features, a code review that includes interfaces, and an account of the problems. The console path is past that line. The product as a whole is a working core with a built GUI and a list of known gaps. It is not a finished faculty system.

Area	Where we are	Evidence
Login and roles	Done and shown	lecturer1 has no user menu. hod1 has option 6.
SQLite storage	Done and shown	23 students, 31 subjects, 123 mark rows. 83.25 survives a restart.
Weighted marks and grades	Done and shown	82, 71 and 90 with weights 25/25/50 become 83.25, B+. GPA 3.25.
Reports and missing marks	Done and shown	Class average 70.75. 20 students flagged as missing.
CSV import and export	Done and shown	40 records written to /tmp/demo_marks.csv.
Validation	Done	Scores must be 0 to 100. Weights must total 100. Bad input is re-prompted.
Interfaces	Done in this build	MarkRepository and AuthenticationService. Callers use those types.
Automated tests	Done for the core	28 tests, 0 failures, from mvn test.
Swing GUI	Built, not in this demo	12 screens: login, dashboard, students, subjects, marks, reports, CSV, users.
Multi-user server	Not started	One local SQLite file. No shared server and no network login.
Production hardening	Not done	See section 5. Known passwords, large classes, CSV columns by number.

The project database `data/marks.db` already holds the seeded class list (23 / 31 / 123 and three users). The recording used a new file. After the recording, `/tmp/demo.db` still has student 253243782 in CIV260S as Test 1 = 82, Test 2 = 71, Exam = 90, and the user passwords start with `pbkdf2$`.

The console demo that this status is based on

From the project folder: `mvn clean package`, then `java -Dmarksystem.db=/tmp/demo.db -jar target/marks-administration-system-1.0.0.jar`, then choice 1. The home-directory Maven error was only a wrong folder: there is no `pom.xml` in the home directory.

```
Logged in as: Dr. John Doe (LECTURER)
CIV260S  Structural Analysis 2
  Test 1  25%    Test 2  25%    Exam  50%

Student 253243782  Hussein Akim
  Test 1 = 82    Test 2 = 71    Exam = 90
  Weighted Total: 83.25     Grade: B+
  Cumulative GPA: 3.25

Subject report: average 70.75, highest 83.25, lowest 57.25
  20 students with missing or absent marks
CSV: Successfully exported 40 record(s)

Second start, same file, no retyping: still 83.25, B+
hod1 login: menu option 6, User Management, three accounts
```

Arithmetic: 82 × 0.25 + 71 × 0.25 + 90 × 0.50 = 83.25. B+ starts at 80. The calculation lives in `Mark.calculateWeightedTotal`. The save lives in `MarkService.enterMarks`, which calls `MarkRepository.saveMark`. `Database` does that write in one transaction: delete the old rows for that student and subject, insert the new ones, commit, or roll back.

3. Issues we hit, and what was fixed in the code

These are problems that showed up while the layers were joined, and that the current source already handles. They are not a wish list.

3.1 The database locked, and a report export deadlocked

The console, the GUI and a background CSV import each opened their own SQLite connection. A second writer failed with “database is locked”. The report export then took the service read lock and, still holding it, wrote an activity-log row, which needs the write lock. That upgrade never completed.

Fixed in `Database.connect`: every connection runs `PRAGMA foreign_keys = ON` and `PRAGMA busy_timeout = 5000`, so a writer waits instead of failing at once. Mark saves and deletes commit or roll back explicitly. The activity log is written after the read lock is released. A CSV import takes `MarkService.withWriteLock` so it cannot interleave with a mark typed in the GUI.

3.2 Renaming an assessment silently ruined existing marks

Weights live on the subject, scores live on the mark, and the two maps were joined by category name. Renaming “Exam” to “Final Exam” left the old score in the database and created an empty new category. Half the weight disappeared. An 83.25 result could fall to 75 with no message.

Fixed in `Mark.alignToCategories`. On a subject edit, and again when marks are loaded from SQLite, the score map is rebuilt from the subject’s current categories. A kept category keeps its score. A new category is missing. A removed category is dropped. `MarkService` and `Database.loadMarks` both call it. A regression test covers the rename.

3.3 A missing exam made the grade look better

The first formula divided only by the weights that had a number. A student who skipped the 50% exam was averaged over the two tests alone and could still pass. The marks screen and the reports also disagreed about what “missing” meant.

Fixed in `calculateWeightedTotal`. The total divides by the subject’s full weight. A null score sets `isMissing` and adds nothing. GPA, the subject statistics and the missing-marks report all read that flag. In the demo, CIV260S reported 20 students with missing or absent marks, using this rule.

3.4 An exported CSV could not be imported again

The export added `WeightedTotal` and `Grade` columns for Excel. The importer treated every column after the name fields as an assessment, so those calculated columns shifted the scores. Blank template rows and a second row for the same student and module were counted as new work, and one bad row aborted the file.

Fixed in `CSVService`. Ordinary columns are matched by header name, so order can change. Assessment columns are the headers `Assessment1`, `Assessment2`, and so on. Calculated columns are ignored and the total is calculated again. Blank rows are skipped. A duplicate student-and-module pair is counted once. A bad row becomes a warning on `ImportSummary` and the rest of the file still loads. The demo export of 40 records uses that layout.

3.5 New password hashes would have locked everyone out

The first database stored salted SHA-256. Moving every account to PBKDF2 would have made the old rows unable to log in. A damaged hash string also threw during login instead of failing the attempt.

Fixed in `PasswordUtils`. A new hash is `pbkdf2$iterations$salt$hash` (120 000 iterations, random salt, constant-time compare). The checker looks at the prefix and still accepts a legacy SHA-256 string. A malformed string returns false. The current user is published under the write lock. Neither the console nor the GUI will delete the account that is logged in. The demo database shows the new prefix on lecturer1, lecturer2 and hod1.

3.6 The console crashed on bad input, and the GUI wiped marks

A year, a credit value or a mark that was not a number used to fall out of `Scanner` and end the program. On the marks screen, opening the form with empty fields and saving one assessment replaced the others with blanks. After an edit, other tabs still showed the old rows. The subject dialog added an empty assessment row. The report export button did nothing.

Fixed in the console by re-prompting until the number is inside the allowed range. Fixed in the GUI by loading the selected student's current scores into the form, rejecting a non-numeric mark, and calling one `refreshAll` callback after a save or an import so every tab reloads. Subject dialogs are rebuilt from the weights that already exist. The export action is wired. `MarkPanelTest` checks the preload without opening a window.

3.7 The layers did not agree, because eight people built them separately

The work was split into four pairs: database and domain model, accounts and the marks engine, the CSV reader and the import service, and the screens plus tests. Each pair could show its own layer. The first full run failed because the pairs did not share one meaning of a missing mark, and because the CSV importer depended on category names that the marks pair was still changing.

What we changed: one SQLite file for the console and the GUI, service methods as the only way into the rules, and tests that save a mark and read it back in a new `MarkService`. File ownership is in `docs/WORK_DIVISION.md`. Each member's share is 12.5%. The two interfaces, `MarkRepository` and `AuthenticationService`, are the contracts that were missing from the first review. They are in this build. The console login and the user screens take `AuthenticationService`. The services take `MarkRepository`.

4. What we still need to fix

The items below are still true in the source we are handing in. They are the next work, not defects in the demo numbers.

Still open	Why it matters	Where it sits
MarkService is 656 lines and owns students, subjects and marks together.	A change to subject weights can break mark entry. The class should become three services over one repository.	services/MarkService.java
Database is 675 lines of schema, SQL and row mapping in one class.	The SQL is hard to review. A mapper per table would leave Database as the connection and the transactions.	db/Database.java
Assessment CSV columns are numbered, not named.	Assessment1 is the first category, Assessment2 the second. Renaming a header in Excel moves scores onto the wrong test. Header names of the other columns are already matched by name.	CSVService.assessmentColumns
The demo accounts use published passwords.	lecturer1 / pass123 and hod1 / admin123 are fine for a marker and unsafe on a shared machine. There is no forced change on first login and no lockout after bad attempts.	UserService.initializeDefaultUsers
GUI tabs refresh through one hand-written callback.	A new tab has to be added to refreshAll by hand or it shows stale rows. A property-change listener would do that from the service.	gui/GUIMain.java
Only the marks panel has a GUI test.	Login, students, subjects, reports, CSV and users are checked by hand. A broken dialog would not fail mvn test.	gui/MarkPanelTest.java is the only GUI test
A large or failed CSV import has no progress and no retry.	The import runs in the background and returns an ImportSummary. The user cannot see progress or repeat only the failed rows.	services/CSVService.java
The console menus are not under JUnit.	mvn test covers the services. The menu path is the scripted demo and the video. A prompt change can break the demo without a red test.	cli/CLIMain.java
Getters on the model classes have no Javadoc.	The brief asks for commented methods. The business methods are documented. The getters are not, on purpose so far, and a strict reading can still mark that down.	models/Student.java, Subject.java, Mark.java, User.java
There is no shared server.	Two people cannot work on one class list unless they share the SQLite file. Concurrent use of one file is protected by the busy timeout, not by a server.	db/Database.java

None of those stop the console demonstration. Login, the 83.25 result, the restart, the CSV export and the HoD menu all work on the current code. The first two rows are the ones we would do before adding another feature, because `MarkService` and `Database` are where the next bug would hide.

5. Class diagram, lined up with the source

The GUI classes are omitted. They exist and they call the same services. This hand-in does not demonstrate them.

```
App --> CLIMain
CLIMain --> AuthenticationService <|.. UserService
CLIMain --> MarkService --> MarkRepository <|.. Database
CLIMain --> ReportService --> MarkService
CLIMain --> CSVService --> MarkService
UserService --> MarkRepository
UserService ..> PasswordUtils
CSVService ..> CSVHelper
MarkService ..> CPUTData
MarkService o-- Student o-- Mark
MarkService o-- Subject
```

Type on the diagram	File	Demo result it produced
App	App.java	Choice 1 starts the console.
CLIMain	cli/CLIMain.java	Menus, prompts, and the HoD gate.
AuthenticationService	services/AuthenticationService.java	The login contract the console uses.
UserService	services/UserService.java	lecturer1 and hod1. Option 6 for the HoD only.
MarkService	services/MarkService.java	Stores 82, 71 and 90.
MarkRepository	db/MarkRepository.java	The save/load contract. The mark is there after restart.
Database	db/Database.java	SQLite. 23 / 31 / 123 after seeding.
ReportService	services/ReportService.java	GPA 3.25. Class average 70.75.
CSVService	services/CSVService.java	40 records exported.
Student, Subject, Mark	models/	Hussein Akim, CIV260S at 25/25/50, result 83.25 B+.
PasswordUtils	utils/PasswordUtils.java	Stored passwords start with pbkdf2$.
CPUTData	utils/CPUTData.java	The 23 students and 31 modules on an empty file.

Repeat the session from the project directory with choice 1, lecturer1 / pass123, subject CIV260S, student 253243782, marks 82, 71 and 90, then the subject report and CSV export. Start the jar again with the same database path to see 83.25 without typing it again. The video is `/Users/husseinakim/Downloads/Group21_CLI_Demo.mp4`.